Building data course — Week 2 — lookups, joins and conditions

sparql-playground / lessons

Generated for print and offline reading

Week 2 — lookups, joins and conditions

Five daily lessons, all run from Python. Each computation is done twice, as a SQLite query string (stdlib `sqlite3`) and in pandas, and `lab.assert_parity` checks they agree. Run a day with `uv run python lessons/course.py run N` (it uses a temporary copy of `var/timeseries.sqlite`; build that with `uv run bldgts bootstrap`). `assert` lines state the answers you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 6 — A point list table (`lab_point`), and why it's a stand-in for the RDF model
- Day 7 — JOIN: readings by role and equipment instead of UUID
- Day 8 — Self-joins: static pressure *while the fan is running*
- Day 9 — CASE and conditional aggregation: run hours, percent occupied
- Day 10 — Time buckets and views: hourly averages (what Timescale's `time_bucket` does)

Day 6 — A point list table (`lab_point`), and why it's a stand-in for the RDF model

Goal: get the point list into the database (and into a DataFrame), so queries can say "duct static pressure" instead of a UUID.

Read

The UUID → meaning mapping lives in `model/points/*.csv`, one file per AHU. Today you flatten both files into one table, `lab_point`. `lab.load_points(conn)` does it (it adds `equipment_id`, which isn't a column in the files; it comes from the file name, `BUILDING_50__AHU_1.csv` → `AHU_1`).

Column	Example	Meaning
`timeseries_id`	`52422584-…`	joins to `samples.timeseries_id`
`point_id`	`AHU_1_DA_P`	a readable name for the point
`equipment_id`	`AHU_1`	which AHU's mapping file it came from
`owner_id`	`AHU_1_Supply_Fan`	the piece of equipment the point belongs to
`brick_class`	`Supply_Air_Static_Pressure_Sensor`	what kind of point it is (Brick vocabulary, Unit 3)
`unit`	`IN_H2O`	engineering unit
`openfdd_role`	`duct-static-pressure`	the role a fault rule asks for, if any
`source_column`	`da_p_inwc`	the column name in the BAS export

Weeks 1 and 2 stay in the raw units the BAS logged: °F, inH2O, and percent 0–100. The SI conversion (°C, Pa, fractions) happens once, on Day 11.

Two ways to hold the same table. In SQL it's a table with a primary key. In pandas it's a DataFrame you build with `pd.read_csv` and `pd.concat`. The same operations have different names:

Question	SQL on `lab_point`	pandas on a DataFrame `pts`
how many rows	`SELECT count(*)`	`len(pts)`
non-NULL count	`count(openfdd_role)`	`pts["openfdd_role"].count()`
distinct count	`count(DISTINCT brick_class)`	`pts["brick_class"].nunique()`
rows per group	`GROUP BY equipment_id`	`pts.groupby("equipment_id").size()`
filter	`WHERE equipment_id = 'AHU_1'`	`pts[pts.equipment_id == "AHU_1"]`
NULL test	`IS NOT NULL`	`.notna()` (never `== None`)

Both treat a missing value as "unknown": SQL's `NULL` and pandas' `NaN` refuse to equal anything, even themselves. That's why SQL needs `IS NOT NULL`, and why `= NULL` matches nothing.

Why the primary key matters. `lab_point.timeseries_id` is the primary key, so SQLite keeps a B-tree on it. Finding one point by UUID is O(log n), not a scan of every row. For 118 points that's invisible, but it is the exact lookup Day 7's join does for each of 924,000 role readings. A DataFrame has no index on a column unless you ask for one (`set_index`); a plain boolean filter scans every row, O(n).

Why this table is a compromise. Look at `owner_id`: the supply fan status belongs to `AHU_1_Supply_Fan`, which is part of `AHU_1`. Zone temperatures belong to `Zone_VAV_107`, which AHU_1 feeds. A flat table holds *one* "belongs to" column. A building is a **graph**: fans are part of AHUs, AHUs feed zones, sensors are points of equipment. You'll be tempted to write `owner_id LIKE 'AHU_1%'`. That works here by luck of the naming and breaks on the next site. The RDF model stores those relationships explicitly (`brick:isPartOf`, `brick:feeds`, `brick:isPointOf`), and Units 3–4 query them with SPARQL. For three weeks of SQL, this table is enough.

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)  # creates lab_point from model/points/*.csv if
         it isn't there yet
```

The point list in pandas, straight from the CSV files. This is the oracle: `lab_point` has to match it.

```
frames = []
for path in lab.POINT_CSVS:
    df = pd.read_csv(path, dtype=str)  # keep everything as text, like
         the table
    df["equipment_id"] = path.stem.split("__", 1)[1]  #
         BUILDING_50__AHU_1 -> AHU_1
    frames.append(df)
pts = pd.concat(frames, ignore_index=True)
pts = pts.astype(object).where(
    pts.notna(), None
)  # NaN -> None, so text NULLs compare equal in parity

COLUMNS = [
    "timeseries_id",
    "point_id",
    "equipment_id",
    "owner_id",
    "label",
    "brick_class",
    "unit",
    "openfdd_role",
    "source_column",
]
lab_point = lab.query(conn, "SELECT * FROM lab_point")
lab.assert_parity(pts[COLUMNS], lab_point, keys=["timeseries_id"],
         label="lab_point vs CSV files")
print(pts[COLUMNS].head(3).to_string(index=False))
```

Same 118 rows, same values. Empty CSV cells come out of `read_csv` as `NaN` and are `NULL` in the table. We turned the `NaN`s into `None` (what `sqlite3` turns `NULL` into) so that missing *text* compares equal; `.notna()` and `.count()` treat both the same.

```
COUNTS_SQL = """
SELECT count(*) AS points, count(openfdd_role) AS with_role,
       count(DISTINCT brick_class) AS classes
FROM lab_point
"""
counts_sql = lab.query(conn, COUNTS_SQL)
counts_pd = pd.DataFrame(
    {
        "points": [len(pts)],
        "with_role": [pts["openfdd_role"].count()],
        "classes": [pts["brick_class"].nunique()],
    }
)
lab.assert_parity(counts_pd, counts_sql, label="counts")
print(counts_sql.iloc[0].to_dict())
assert counts_sql.iloc[0].tolist() == [118, 26, 30]
```

`count(openfdd_role)` skips NULLs, so 26 of the 118 points have a fault-rule role: 13 per AHU.

```
PER_EQUIP_SQL = """
SELECT equipment_id, count(*) AS points
FROM lab_point
GROUP BY equipment_id
ORDER BY equipment_id
"""
per_equip_sql = lab.query(conn, PER_EQUIP_SQL)
per_equip_pd =
        pts.groupby("equipment_id").size().rename("points").reset_index()
lab.assert_parity(per_equip_pd, per_equip_sql, label="points per AHU")
print(per_equip_sql.to_string(index=False))
assert per_equip_sql.set_index("equipment_id")["points"].to_dict() ==
        {"AHU_1": 83, "AHU_2": 35}
```

```
ROLES_SQL = """
SELECT openfdd_role, point_id, unit, timeseries_id
FROM lab_point
WHERE equipment_id = :equip AND openfdd_role IS NOT NULL
ORDER BY openfdd_role
"""
roles_sql = lab.query(conn, ROLES_SQL, {"equip": "AHU_1"})
sel = pts[(pts.equipment_id == "AHU_1") & pts.openfdd_role.notna()]
lab.assert_parity(
    sel[["openfdd_role", "point_id", "unit",
         "timeseries_id"]].reset_index(drop=True),
    roles_sql,
    keys=["openfdd_role"],
    label="AHU_1 roles",
)
print(roles_sql[["openfdd_role", "point_id",
         "unit"]].to_string(index=False))
assert len(roles_sql) == 13
```

`IS NOT NULL` in SQL, `.notna()` in pandas. Never `!= NULL` or `!= None`: a comparison with a missing value is never true.

Is `lab_point` really the model? The app's RDF model (`model/building_50.ttl`) was built from these same CSV files. Ask its SPARQL endpoint for every point that has an `ofddRole` tag: which equipment it belongs to (`equipRef`) and its timeseries UUID. `lab.Api` runs the repo's FastAPI app in-process (no server, same database), and `api.sparql` returns a DataFrame. Equipment comes back as an IRI, so we keep the local name after `#`.

```
MODEL_QUERY = """
PREFIX bts: <https://example.org/openfdd/bldgts#>
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?equip ?role ?timeseriesId WHERE {
  ?point ref:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?timeseriesId .
  ?point bts:hasHaystackTag ?roleTag .
  ?roleTag bts:tagName "ofddRole" ; bts:tagValue ?role .
  ?point bts:hasHaystackTag ?direct .
  ?direct bts:tagName "equipRef" ; bts:tagValue ?equip .
}
"""
with lab.Api() as api:
    graph = api.sparql(MODEL_QUERY)
graph["equipment_id"] = graph["equip"].str.split("#").str[-1]  #
        .../BUILDING_50#AHU_1 -> AHU_1
graph = graph.rename(columns={"role": "openfdd_role", "timeseriesId":
        "timeseries_id"})
graph = graph[["equipment_id", "openfdd_role", "timeseries_id"]]

TRIPLE_SQL = """
SELECT equipment_id, openfdd_role, timeseries_id FROM lab_point WHERE
        openfdd_role IS NOT NULL
"""
from_sql = lab.query(conn, TRIPLE_SQL)
from_csv = pts.loc[
    pts.openfdd_role.notna(), ["equipment_id", "openfdd_role",
        "timeseries_id"]
].reset_index(drop=True)

lab.assert_parity(from_csv, from_sql, label="CSV files vs lab_point
        (SQL)")
lab.assert_parity(from_csv, graph, label="CSV files vs the RDF model
        (SPARQL)")
assert len(graph) == 26
```

Three routes to the same 26 `(equipment, role, UUID)` facts: the CSV files read with pandas, the SQL table, and a SPARQL query over the graph. That is the point of today: `lab_point` is a flat copy of what the model says. What the flat copy can't hold is the structure around those facts (fans *part of* AHUs, AHUs *feeding* zones), and that's what SPARQL adds.

The order of the patterns in `MODEL_QUERY` is deliberate. rdflib evaluates them roughly top to bottom, like a nested loop join (Day 7). Starting from `ref:hasExternalReference` binds only the 118 points that have history, so every later pattern is a cheap lookup. Starting from "every tag named `ofddRole`" first is the same answer but a much bigger intermediate result, slow enough to hit the API's 10-second query timeout. A database's query planner reorders joins for you; a simple SPARQL engine may not.

(One performance note. The patterns start from points that have a timeseries UUID, the 118 selective ones. With the same patterns listed in another order the query took over 10 seconds and the API cut it off with a

504. A graph engine, like SQLite's join planner on Day 8, is sensitive to where it starts.)

What the primary key buys. Look up one point by UUID three ways: SQL through the primary-key index, a pandas boolean filter (scans every row), and a pandas lookup after `set_index` (a hash table). With 118 rows all three are microseconds, so repeat the table 2,000 times to make the scan visible.

```
big = pd.concat([pts] * 2000, ignore_index=True)  # 236,000 rows, 118
        distinct UUIDs
target = pts.timeseries_id.iloc[-1]
by_uuid = big.set_index("timeseries_id").sort_index()  # a sorted index
        on the column

t_scan = lab.best_time(lambda: big[big.timeseries_id == target])
t_index = lab.best_time(lambda: by_uuid.loc[[target]])
t_sql = lab.best_time(
    lambda: conn.execute("SELECT * FROM lab_point WHERE timeseries_id =
        ?", (target,)).fetchall()
)
print(
    f"pandas scan {t_scan * 1e6:8.0f} us | pandas index {t_index *
        1e6:8.0f} us | SQL primary key {t_sql * 1e6:6.0f} us"
)
assert t_sql < t_scan  # an index lookup beats scanning 236,000 rows
```

The scan touches every row; the indexed lookups don't. SQLite's table is tiny, but its lookup cost would barely change at 118 million rows (a B-tree grows one level per ~100× more rows). That is why Day 7's join can afford to look up a point for every reading.

Try it

1. What is AHU_2's duct static pressure UUID?
2. Why does AHU_1 have 83 points but AHU_2 only 35? Look at `brick_class`.

Answers

```
UUID_SQL = """
SELECT timeseries_id FROM lab_point
WHERE equipment_id = 'AHU_2' AND openfdd_role = 'duct-static-pressure'
"""
uuid_sql = lab.query(conn, UUID_SQL)
mask = (pts.equipment_id == "AHU_2") & (pts.openfdd_role == "duct-
        static-pressure")
lab.assert_parity(
    pts.loc[mask, ["timeseries_id"]].reset_index(drop=True), uuid_sql,
        label="AHU_2 static UUID"
)
assert uuid_sql.iloc[0, 0] == "f8a0d594-2fff-567a-9853-71ba66233eba"
print(uuid_sql.iloc[0, 0])
```

```
ZONE_SQL = """
SELECT equipment_id, count(*) AS zone_temps FROM lab_point
WHERE brick_class = 'Zone_Air_Temperature_Sensor'
GROUP BY equipment_id
"""
zone_sql = lab.query(conn, ZONE_SQL)
zones = pts[pts.brick_class == "Zone_Air_Temperature_Sensor"]
zone_pd =
         zones.groupby("equipment_id").size().rename("zone_temps").reset_index()
lab.assert_parity(zone_pd, zone_sql, label="zone temperature sensors")
print(zone_sql.to_string(index=False))
assert zone_sql.set_index("equipment_id")["zone_temps"].to_dict() ==
         {"AHU_1": 47}
```

AHU_1's export included the zone temperatures of the VAV boxes it serves (47 of them). AHU_2's didn't, so it has no row at all in the result. Same kind of building, different export: a point list tells you what was **exported**, not what exists.

Why it matters

`lab_point` is a lookup table: id → attributes. In week 5 you'll build the same thing as a dict in Python (a hash table, O(1) per lookup, like the pandas index above), then as a set of triples `(point, hasUnit, IN_H2O)`. That's the shape of RDF.

Day 7 — JOIN: readings by role and equipment instead of UUID

Goal: combine `samples` and `lab_point` so you can ask for readings by meaning, in SQL and with a pandas merge.

Read

A **JOIN** pairs rows from two tables that match a condition:

```
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id = s.timeseries_id
```

For every reading `s`, find the point row `p` with the same `timeseries_id`, and glue them into one wider row. `s` and `p` are **aliases** (short names), so you can write `s.value` and `p.unit`.

- `JOIN` (inner join) keeps only rows that matched.
- `LEFT JOIN` keeps every row from the left table and fills the right side with NULLs when nothing matched. It's the tool for "which readings have no point?" style checks.

Once joined, you filter and group by point attributes: `WHERE p.openfdd_role = 'fan-status'`, `GROUP BY p.equipment_id`. No UUIDs typed.

In pandas the same thing is `samples.merge(pts, on="timeseries_id", how="inner")`, and `how="left"` is the `LEFT JOIN`.

	SQL	pandas
inner join	`JOIN lab_point p ON p.timeseries_id = s.timeseries_id`	`s.merge(p, on="timeseries_id")`
left join	`LEFT JOIN ...`	`s.merge(p, on="timeseries_id", how="left", indicator=True)`
filter after join	`WHERE p.openfdd_role = 'fan-status'`	`m[m.openfdd_role == "fan-status"]`
aggregate	`GROUP BY p.equipment_id` + `sum(...)`	`m.groupby("equipment_id")["value"].sum()`
"this must be one-to-one"	a `PRIMARY KEY` / `UNIQUE` constraint	`merge(..., validate="m:1")`

How the join is computed. The two tools use different algorithms.

- SQLite does a **nested loop with an index lookup**. It picks an outer table, and for each of its rows it seeks the matching row in the inner table through an index. That costs O(n log m) for n outer rows and m inner rows, and needs almost no extra memory. This is why `samples` has `PRIMARY KEY (timeseries_id, ts)` and `lab_point` has `PRIMARY KEY (timeseries_id)`: those are the indexes the seeks use. It also lets SQLite pick the *cheaper* outer table. With `WHERE openfdd_role = 'fan-status'` it starts from the two fan-status points and seeks only their readings, never touching the other 4 million rows.
- pandas does a **hash join**. It builds a hash table of one side's keys and probes it with the other side's rows: O(n + m) time, but the data has to be in memory (a DataFrame for each side plus the result), and the merge happens on whatever you loaded. You do the "pick only the fan-status readings" step yourself, before the merge.

Join explosion. If the key isn't unique on the side you're looking up, every match makes its own output row, and the result has *more* rows than the readings you started with. Sums and averages quietly turn wrong. Always check that a join keeps the row count you expect (SQL: look at `count(*)`; pandas: `validate="m:1"`).

Units: weeks 1 and 2 use the raw values (°F, inH2O, percent 0–100).

Run

```python
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)
pts = lab.query(conn, "SELECT * FROM lab_point")  # 118 rows: the
        lookup side of every join


def role_samples(roles):
    """Readings of just the points that carry one of these roles (not
        all 4.2 million rows), as a DataFrame."""
    ids = pts.loc[pts.openfdd_role.isin(roles),
        "timeseries_id"].tolist()
    marks = ",".join("?" * len(ids))
    return lab.query(
        conn, f"SELECT timeseries_id, ts, value FROM samples WHERE
         timeseries_id IN ({marks})", ids
    )
```

`role_samples` keeps pandas memory sensible: it asks SQLite for only the readings it needs, rather than loading the whole table and filtering afterwards.

Supply fan run hours for every AHU, no UUIDs typed. Fan status is 1 when running, 0 when off, one reading per 5 minutes, so `sum(value) * 5 / 60` is hours.

```
RUN_HOURS_SQL = """
SELECT p.equipment_id, sum(s.value) * 5 / 60.0 AS run_hours
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id = s.timeseries_id
WHERE p.openfdd_role = 'fan-status'
GROUP BY p.equipment_id
ORDER BY p.equipment_id
"""
run_sql = lab.query(conn, RUN_HOURS_SQL)

fan = role_samples(["fan-status"]).merge(
    pts[["timeseries_id", "equipment_id"]], on="timeseries_id",
        how="inner", validate="m:1"
)
run_pd = (fan.groupby("equipment_id")["value"].sum() * 5 /
        60).rename("run_hours").reset_index()
lab.assert_parity(run_pd, run_sql, label="fan run hours")
print(run_sql.round(1).to_string(index=False))
assert round(run_sql.set_index("equipment_id").run_hours["AHU_1"], 1)
        == 1574.4
```

`validate="m:1"` says "many readings to one point". If two point rows ever shared a `timeseries_id` the merge would raise instead of silently doubling the answer.

```
AVG_SQL = """
SELECT p.equipment_id, p.openfdd_role, avg(s.value) AS avg_f
FROM samples AS s
JOIN lab_point AS p ON p.timeseries_id = s.timeseries_id
WHERE p.openfdd_role IN ('discharge-air-temp', 'return-air-temp')
GROUP BY p.equipment_id, p.openfdd_role
ORDER BY p.equipment_id, p.openfdd_role
"""
avg_sql = lab.query(conn, AVG_SQL)

temps = role_samples(["discharge-air-temp", "return-air-temp"]).merge(
    pts[["timeseries_id", "equipment_id", "openfdd_role"]],
        on="timeseries_id", validate="m:1"
)
avg_pd = (
    temps.groupby(["equipment_id", "openfdd_role"])
        ["value"].mean().rename("avg_f").reset_index()
)
lab.assert_parity(avg_pd, avg_sql, label="average discharge vs return")
print(avg_sql.round(2).to_string(index=False))
row = avg_sql[(avg_sql.equipment_id == "AHU_1") & (avg_sql.openfdd_role
        == "discharge-air-temp")]
assert round(row.avg_f.iloc[0], 2) == 61.17
```

Integrity check: any readings whose UUID is not in the point list? In SQL, a `LEFT JOIN` and look for the NULLs. In pandas, a left merge with `indicator=True` adds a column saying which side each row came from. Counting readings means reading all 4.2 million, so first collapse them to one row per series (`GROUP BY` in SQL, before the merge). Aggregating before joining is the cheap order, in both tools.

```
ORPHANS_SQL = """
SELECT count(*) AS orphan_readings
FROM samples AS s
LEFT JOIN lab_point AS p ON p.timeseries_id = s.timeseries_id
WHERE p.timeseries_id IS NULL
"""
orphans_sql = lab.query(conn, ORPHANS_SQL)

per_series = lab.query(
    conn, "SELECT timeseries_id, count(*) AS n FROM samples GROUP BY
         timeseries_id"
)
checked = per_series.merge(pts[["timeseries_id"]], on="timeseries_id",
         how="left", indicator=True)
orphans_pd = pd.DataFrame(
    {"orphan_readings": [checked.loc[checked["_merge"] == "left_only",
         "n"].sum()]}
)
lab.assert_parity(orphans_pd, orphans_sql, label="orphan readings")
assert orphans_sql.iloc[0, 0] == 0
print(len(per_series), "series in samples,", len(pts), "points in
         lab_point")
```

Zero orphans. The repo's tests enforce the same invariant on the RDF model: every series in the store has a point, and every point's series is in the store.

Join explosion, on purpose. Duplicate the two fan-status point rows, as a sloppy export might, and join again. Row counts double, and so does every sum.

```
fan_ids = pts.loc[pts.openfdd_role == "fan-status", ["timeseries_id",
          "equipment_id"]]
dup_pts = pd.concat(
    [pts[["timeseries_id", "equipment_id"]], fan_ids]
)  # 120 rows, 2 UUIDs appear twice

conn.execute("DROP TABLE IF EXISTS temp.dup_point")
conn.execute(
    "CREATE TEMP TABLE dup_point (timeseries_id TEXT, equipment_id
          TEXT)"
)  # no primary key
conn.executemany("INSERT INTO dup_point VALUES (?, ?)",
          dup_pts.values.tolist())

n_fan = len(fan)
n_sql = conn.execute(
    """SELECT count(*) FROM samples s JOIN dup_point p ON
          p.timeseries_id = s.timeseries_id
       WHERE s.timeseries_id IN (SELECT timeseries_id FROM lab_point
         WHERE openfdd_role = 'fan-status')"""
).fetchone()[0]
n_pd = len(role_samples(["fan-status"]).merge(dup_pts,
          on="timeseries_id"))
print(f"fan readings {n_fan} -> SQL join {n_sql} rows, pandas merge
          {n_pd} rows")
assert n_sql == n_pd == 2 * n_fan

try:
    role_samples(["fan-status"]).merge(dup_pts, on="timeseries_id",
          validate="m:1")
except pd.errors.MergeError as err:
    print("validate caught it:", str(err)[:70])
else:
    raise AssertionError("expected a MergeError")
```

What the planner does. `EXPLAIN QUERY PLAN` shows the nested loop SQLite chose for the run-hours query.

```
plan = conn.execute("EXPLAIN QUERY PLAN " + RUN_HOURS_SQL).fetchall()
for step in plan:
    print(step[-1])
```

It scans `lab_point` (118 rows) first, because of the `WHERE` on role, then seeks `samples` through its primary key for each matching point. Now time it against the pandas version. The pandas side is fast too once it has loaded only fan readings, but the work of *choosing* what to load was done by hand.

```
def pandas_run_hours():
    f = role_samples(["fan-status"]).merge(
        pts[["timeseries_id", "equipment_id"]], on="timeseries_id"
    )
    return f.groupby("equipment_id")["value"].sum()


t_sql = lab.best_time(lambda: conn.execute(RUN_HOURS_SQL).fetchall())
t_pd = lab.best_time(pandas_run_hours)
print(f"SQL join {t_sql * 1000:6.1f} ms | pandas load + merge {t_pd *
        1000:6.1f} ms")
```

Neither wins by principle: both read the same 71,072 readings. The SQL cost model is O(n log m) with no big intermediate result; the pandas one is O(n + m) but holds the readings and the merged copy in memory at once. Join the *whole* `samples` table to `lab_point` in pandas and that's 4.2 million rows × several columns, many hundreds of MB. Filter or aggregate first.

Try it

1. Average duct static pressure **setpoint** per AHU. Are the two AHUs controlled to the same setpoint?
2. Count readings per `unit`. Which unit has the most data?

Answers

```
SP_SQL = """
SELECT p.equipment_id, avg(s.value) AS avg_sp FROM samples s
JOIN lab_point p ON p.timeseries_id = s.timeseries_id
WHERE p.openfdd_role = 'duct-static-pressure-sp'
GROUP BY p.equipment_id ORDER BY p.equipment_id
"""
sp_sql = lab.query(conn, SP_SQL)
sp = role_samples(["duct-static-pressure-sp"]).merge(
    pts[["timeseries_id", "equipment_id"]], on="timeseries_id",
        validate="m:1"
)
sp_pd = sp.groupby("equipment_id")
        ["value"].mean().rename("avg_sp").reset_index()
lab.assert_parity(sp_pd, sp_sql, label="average setpoint")
print(sp_sql.round(3).to_string(index=False))
assert round(sp_sql.set_index("equipment_id").avg_sp["AHU_1"], 3) ==
        1.366
```

AHU_1 averages 1.37 inH2O, AHU_2 0.78. Different setpoints, which is normal: each fan system is sized for its own ductwork.

```
UNIT_SQL = """
SELECT coalesce(p.unit, '(none)') AS unit, count(*) AS readings
FROM samples s JOIN lab_point p ON p.timeseries_id = s.timeseries_id
GROUP BY unit ORDER BY readings DESC
"""
unit_sql = lab.query(conn, UNIT_SQL)

# pandas: aggregate to one row per series first (35,536 readings each),
          then merge the 118-row table
unit_pd = (
    per_series.merge(pts[["timeseries_id", "unit"]],
         on="timeseries_id", validate="1:1")
    .assign(unit=lambda d: d["unit"].fillna("(none)"))
    .groupby("unit")["n"]
    .sum()
    .rename("readings")
    .reset_index()
)
lab.assert_parity(unit_pd, unit_sql, label="readings per unit")
print(unit_sql.to_string(index=False))
assert unit_sql.iloc[0].tolist() == ["DEG_F", 2380912]
```

`coalesce(x, y)` returns the first non-NULL argument; in pandas that's `fillna`. Here it labels the points that have no unit (statuses, commands, alarms). The pandas version never merges 4.2 million readings: it merges 118 per-series counts, which is the aggregate-before-join order.

Why it matters

"Join readings to metadata, filter by metadata" is exactly what the API does. The difference is that its metadata comes from a SPARQL query over the graph rather than a table. Week 12's client runs a SPARQL query, gets UUIDs back, and reads samples. Same join, across two systems.

Day 8 — Self-joins: static pressure *while the fan is running*

Goal: line up two different points at the same timestamp, and filter one by the other, in SQL and in pandas.

Read

In long format, the fan status and the duct static pressure at 10:05 are **two different rows** of `samples`. To ask "what was static pressure while the fan ran?", you need them side by side in one row. That takes a **self-join**: join `samples` to itself, once for each point, matching on the same `ts`.

```
fan status reading  f   ─┐
                         ├─ same ts, same AHU ─▶ one row: (ts, fan
status, static)
static reading      d   ─┘
```

Each copy of `samples` needs its own `lab_point` copy to pick the role:

```
FROM lab_point fp JOIN samples f  ON f.timeseries_id = fp.timeseries_id
-- fan status readings
JOIN lab_point dp ON dp.equipment_id = fp.equipment_id
                 AND dp.openfdd_role = 'duct-static-pressure'
-- same AHU's static point
JOIN samples d    ON d.timeseries_id = dp.timeseries_id AND d.ts = f.ts
-- its reading at the same time
WHERE fp.openfdd_role = 'fan-status'
```

Matching on `ts` works because this historian logs every point on the same 5-minute grid. If two points were logged at different times, you'd first round them onto a common grid (Day 10).

The same thing in pandas. There are two natural ways:

Idea	SQL	pandas
one frame per role, join them	`JOIN samples d ON d.ts = f.ts AND ...`	`f.merge(d, on=["equipment_id", "ts"])`
reshape long → wide once	(Day 11: `MAX(CASE WHEN role = ...)`)	`long.pivot(index=[...], columns="openfdd_role", values="value")`
filter by the other point	`WHERE f.value = 1`	`m[m.fan == 1]`

Join algorithms, again. SQLite runs the self-join as a nested loop. Its outer loop is the 71,072 fan readings. For each one it seeks the static reading and the setpoint reading at the same `ts` through the `PRIMARY KEY (timeseries_id, ts)` of `samples`. The key is *both* columns, so each seek lands

on exactly one row (O(log n)), and the self-join cannot multiply rows. pandas does each `merge` as a hash join on `(equipment_id, ts)`: O(n + m) per merge, with a new DataFrame after each one. `pivot` does the reshaping in one pass and raises an error if any `(equipment_id, ts)` pair appears twice, which is the same safety net as the primary key.

`validate="1:1"` on a merge is the pandas version of that guarantee: if either side had duplicate keys the join would multiply rows, and `validate` raises instead.

Units: raw (°F, inH2O, percent 0–100). Fan status is 0/1.

Run

```
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)
pts = lab.query(conn, "SELECT * FROM lab_point")
ROLE_COLS = pts[["timeseries_id", "equipment_id", "openfdd_role"]]


def role_samples(roles):
    """Readings of just the points that carry one of these roles,
         labelled with equipment and role."""
    ids = pts.loc[pts.openfdd_role.isin(roles),
         "timeseries_id"].tolist()
    marks = ",".join("?" * len(ids))
    raw = lab.query(
        conn, f"SELECT timeseries_id, ts, value FROM samples WHERE
         timeseries_id IN ({marks})", ids
    )
    return raw.merge(ROLE_COLS, on="timeseries_id",
         validate="m:1").drop(columns="timeseries_id")
```

Duct static vs setpoint, only while the supply fan runs.

```
RUNNING_SQL = """
SELECT fp.equipment_id,
       count(*)       AS running_readings,
       avg(d.value)   AS avg_static,
       avg(sp.value)  AS avg_setpoint
FROM lab_point fp
JOIN samples f    ON f.timeseries_id = fp.timeseries_id
JOIN lab_point dp ON dp.equipment_id = fp.equipment_id AND
         dp.openfdd_role = 'duct-static-pressure'
JOIN samples d    ON d.timeseries_id = dp.timeseries_id AND d.ts = f.ts
JOIN lab_point spp ON spp.equipment_id = fp.equipment_id AND
         spp.openfdd_role = 'duct-static-pressure-sp'
JOIN samples sp   ON sp.timeseries_id = spp.timeseries_id AND sp.ts =
         f.ts
WHERE fp.openfdd_role = 'fan-status' AND f.value = 1
GROUP BY fp.equipment_id
ORDER BY fp.equipment_id
"""
running_sql = lab.query(conn, RUNNING_SQL)
print(running_sql.round(3).to_string(index=False))
```

In pandas, take each role as its own frame and merge on equipment and timestamp:

```
long = role_samples(["fan-status", "duct-static-pressure", "duct-
        static-pressure-sp"])


def one(role, name):
    return long.loc[long.openfdd_role == role, ["equipment_id", "ts",
        "value"]].rename(
        columns={"value": name}
    )


KEYS = ["equipment_id", "ts"]
merged = (
    one("fan-status", "fan")
    .merge(one("duct-static-pressure", "static"), on=KEYS,
        validate="1:1")
    .merge(one("duct-static-pressure-sp", "setpoint"), on=KEYS,
        validate="1:1")
)
assert len(merged) == len(
    one("fan-status", "fan")
)  # a clean join keeps every fan row: no loss, no explosion

running = merged[merged.fan == 1]
running_pd = (
    running.groupby("equipment_id")
    .agg(
        running_readings=("ts", "size"),
        avg_static=("static", "mean"),
        avg_setpoint=("setpoint", "mean"),
    )
    .reset_index()
)
lab.assert_parity(running_pd, running_sql, label="static while running
        (merge)")

ahu1 = running_sql.set_index("equipment_id").loc["AHU_1"]
assert (ahu1.running_readings, round(ahu1.avg_static, 3),
        round(ahu1.avg_setpoint, 3)) == (
    18893,
    1.347,
    1.402,
)
```

Compare with Day 4. Over all hours AHU_1's static averaged 0.78 against a 1.37 setpoint, which looked like a fault. While running it's 1.35 against 1.40: close, slightly low. AHU_2 runs at **1.40 against a 0.76 setpoint**, well above. That's worth a look, and it's what a "duct static too high" rule would flag.

The second way is one `pivot`: reshape the long frame to wide once, then every later question is a column filter.

```
wide = long.pivot(
    index=KEYS, columns="openfdd_role", values="value"
).reset_index()  # raises on duplicate keys
wide.columns.name = None
run_w = wide[wide["fan-status"] == 1]
running_wide = (
    run_w.groupby("equipment_id")
    .agg(
        running_readings=("ts", "size"),
        avg_static=("duct-static-pressure", "mean"),
        avg_setpoint=("duct-static-pressure-sp", "mean"),
    )
    .reset_index()
)
lab.assert_parity(running_wide, running_sql, label="static while
        running (pivot)")
```

Both pandas routes match the six-way SQL join. Look at how SQLite decided to run it:

```
for step in conn.execute("EXPLAIN QUERY PLAN " +
        RUNNING_SQL).fetchall():
    print(step[-1])
```

Every `samples` step reads `SEARCH ... USING PRIMARY KEY`: an index seek, never a scan of `samples`. And time the three approaches. The pandas times include loading the three roles from SQLite, so it's the whole job in each case.

```
def merge_way():
    lg = role_samples(["fan-status", "duct-static-pressure", "duct-
        static-pressure-sp"])
    f = lg.loc[lg.openfdd_role == "fan-status", KEYS + ["value"]]
    d = lg.loc[lg.openfdd_role == "duct-static-pressure", KEYS +
        ["value"]]
    m = f.merge(d, on=KEYS, suffixes=("_fan", "_static"))
    return m[m.value_fan == 1].groupby("equipment_id")
        ["value_static"].mean()


def pivot_way():
    lg = role_samples(["fan-status", "duct-static-pressure", "duct-
        static-pressure-sp"])
    w = lg.pivot(index=KEYS, columns="openfdd_role", values="value")
    return w[w["fan-status"] == 1].groupby("equipment_id")["duct-
        static-pressure"].mean()


t_sql = lab.best_time(lambda: conn.execute(RUNNING_SQL).fetchall())
t_merge, t_pivot = lab.best_time(merge_way), lab.best_time(pivot_way)
print(
    f"SQL 6-way join {t_sql:5.2f} s | pandas merges {t_merge:5.2f} s |
        pandas pivot {t_pivot:5.2f} s"
)
```

The shape of the cost matters more than who wins on this machine. SQL does roughly 71,000 × 2 index seeks and keeps nothing in memory. The pandas versions load about 213,000 readings, then hash-join or reshape them in memory: fast, but bounded by RAM. Day 11 does the pivot once, in SQL, and stores the wide table.

Try it

1. Average discharge air temperature on AHU_1 while the fan runs vs while it's off. Why is the minimum while running 0.0 °F?
2. Rewrite the first query for the fan **off** (`f.value = 0`). What's static pressure doing then?

Answers

```
DAT_SQL = """
SELECT f.value AS fan_status, avg(d.value) AS avg_dat, min(d.value) AS
         min_dat
FROM lab_point fp
CROSS JOIN lab_point dp ON dp.equipment_id = fp.equipment_id AND
         dp.openfdd_role = 'discharge-air-temp'
CROSS JOIN samples f    ON f.timeseries_id = fp.timeseries_id
CROSS JOIN samples d    ON d.timeseries_id = dp.timeseries_id AND d.ts
         = f.ts
WHERE fp.openfdd_role = 'fan-status' AND fp.equipment_id = :equip
GROUP BY f.value ORDER BY f.value
"""
dat_sql = lab.query(conn, DAT_SQL, {"equip": "AHU_1"})

dat = role_samples(["fan-status", "discharge-air-temp"])
dat = dat[dat.equipment_id == "AHU_1"]
fan_dat = dat.loc[dat.openfdd_role == "fan-status", ["ts",
         "value"]].rename(
    columns={"value": "fan_status"}
)
dat_dat = dat.loc[dat.openfdd_role == "discharge-air-temp", ["ts",
         "value"]].rename(
    columns={"value": "dat"}
)
joined = fan_dat.merge(dat_dat, on="ts", validate="1:1")
dat_pd = joined.groupby("fan_status")["dat"].agg(avg_dat="mean",
         min_dat="min").reset_index()
lab.assert_parity(dat_pd, dat_sql, label="DAT by fan status")
print(dat_sql.round(1).to_string(index=False))
by_fan = dat_sql.set_index("fan_status")
assert (round(by_fan.avg_dat[1], 1), round(by_fan.min_dat[1], 1)) ==
         (53.3, 0.0)
assert (round(by_fan.avg_dat[0], 1), round(by_fan.min_dat[0], 1)) ==
         (70.1, 45.4)
```

About `CROSS JOIN`: in SQLite it means "join, in exactly this order". The planner is normally free to reorder joins, and for this query it chose to build a temporary index on `d.ts` over four million rows. That takes minutes. Written as above, the order is point, point, fan readings, then the discharge

reading, so every `samples` step is a primary-key seek and the query takes a fraction of a second. Join order is a big part of what an optimizer decides, and when it guesses wrong, you can still pin it. Check with `EXPLAIN QUERY PLAN`.

Running averages 53.3 °F, off 70.1 °F (the air in the duct drifts toward room temperature). A real discharge temperature of 0 °F is impossible. It's a dropout: the controller or historian wrote 0 when it had no value. Treat 0 in a temperature column as **missing**, not cold. Fault rules must not fire on it.

```
OFF_SQL = RUNNING_SQL.replace("f.value = 1", "f.value = 0")
off_sql = lab.query(conn, OFF_SQL)

off_pd = (
    merged[merged.fan == 0]
    .groupby("equipment_id")
    .agg(
        running_readings=("ts", "size"),
        avg_static=("static", "mean"),
        avg_setpoint=("setpoint", "mean"),
    )
    .reset_index()
)
lab.assert_parity(off_pd, off_sql, label="static while fan off")
print(off_sql.round(3).to_string(index=False))
assert len(off_sql) == 2 and (off_sql.avg_static < 0.2).all()
```

Near zero, as it should be with the fan off. That's why the Day 4 averages were misleading. (The `running_readings` column here really counts *off* readings.)

Why it matters

Every fault rule is a condition on several points at the same time, gated by "is the equipment on?". Six joins to line up three points is clumsy, though. On Day 11 you'll pivot the data once into a wide table (the `pivot` above) and write rules as simple `WHERE` clauses.

Day 9 — CASE and conditional aggregation: run hours, percent occupied

Goal: compute several different counts in one pass using `CASE` inside aggregates, and the pandas version with boolean masks and `np.where`.

Read

`CASE` is SQL's if/else, and it returns a value:

```
CASE WHEN condition THEN a
     WHEN other     THEN b
     ELSE c END
```

Put it inside an aggregate and you get **conditional aggregation**: one scan, many answers.

```
sum(CASE WHEN fan = 1 AND occ = 0 THEN 1 ELSE 0 END)   -- readings:
running while unoccupied
avg(CASE WHEN fan = 1 THEN static END)                 -- average
static only while running (ELSE → NULL, ignored)
```

In SQLite a comparison is already 0 or 1, so `sum(fan = 1 AND occ = 0)` works too. `CASE` is clearer and portable to other databases.

The `occupied` role is the AHU's occupancy command: 1 when the schedule says the building is in use. An AHU running while **unoccupied** is one of the most common energy findings in retro-commissioning (RCx).

The same ideas in pandas. A condition on whole columns gives a **boolean mask**, and `True` counts as 1:

SQL	pandas
`CASE WHEN c THEN 1 ELSE 0 END`	`c.astype(int)`, or `np.where(c, 1, 0)`
`sum(CASE WHEN c THEN 1 ELSE 0 END)`	`c.sum()`
`avg(CASE WHEN c THEN x END)` (NULL skipped)	`x[c].mean()`, or `x.where(c).mean()` (NaN skipped)
`CASE WHEN a THEN 'x' WHEN b THEN 'y' ELSE 'z' END`	`np.select([a, b], ["x", "y"], default="z")`
`GROUP BY equipment_id`	`.groupby("equipment_id")`

There is one real difference in cost. SQL's `sum(CASE ...)` is evaluated **row by row inside the scan**, keeping only running totals: O(n) time, O(1) extra memory, however many conditions you stack in one query. pandas builds each mask as a full-length boolean array first (n bytes each), so many masks

cost O(n) memory each. In exchange, a mask is a tight compiled loop, far faster than writing the same `if` in Python for every row. Writing the `CASE` as `df.apply(lambda row: ..., axis=1)` is the slow way, and we time it below.

Units: raw (°F, inH2O, percent 0–100); the status and occupancy points are 0/1.

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)
pts = lab.query(conn, "SELECT * FROM lab_point")
ROLE_COLS = pts[["timeseries_id", "equipment_id", "openfdd_role"]]
KEYS = ["equipment_id", "ts"]


def role_samples(roles):
    """Readings of just the points that carry one of these roles,
         labelled with equipment and role."""
    ids = pts.loc[pts.openfdd_role.isin(roles),
         "timeseries_id"].tolist()
    marks = ",".join("?" * len(ids))
    raw = lab.query(
        conn, f"SELECT timeseries_id, ts, value FROM samples WHERE
         timeseries_id IN ({marks})", ids
    )
    return raw.merge(ROLE_COLS, on="timeseries_id",
         validate="m:1").drop(columns="timeseries_id")


def wide_roles(roles):
    """One row per (equipment, ts), one column per role: the long ->
         wide pivot from Day 8."""
    w = role_samples(roles).pivot(index=KEYS, columns="openfdd_role",
         values="value").reset_index()
    w.columns.name = None
    return w
```

Hours each AHU ran while unoccupied, and was off while occupied. The SQL needs a join per point (Day 8). The pandas version pivots once, then the three answers are three masks over the same columns.

```
UNOCC_SQL = """
SELECT fp.equipment_id,
       sum(CASE WHEN f.value = 1 AND o.value = 0 THEN 1 ELSE 0 END) * 5
         / 60.0 AS run_unocc_h,
       sum(CASE WHEN f.value = 0 AND o.value = 1 THEN 1 ELSE 0 END) * 5
         / 60.0 AS off_occ_h,
       100.0 * avg(o.value)
         AS pct_occupied
FROM lab_point fp
CROSS JOIN lab_point op ON op.equipment_id = fp.equipment_id AND
         op.openfdd_role = 'occupied'
CROSS JOIN samples f    ON f.timeseries_id = fp.timeseries_id
CROSS JOIN samples o    ON o.timeseries_id = op.timeseries_id AND o.ts
         = f.ts
WHERE fp.openfdd_role = 'fan-status'
GROUP BY fp.equipment_id
ORDER BY fp.equipment_id
"""
unocc_sql = lab.query(conn, UNOCC_SQL)

w = wide_roles(["fan-status", "occupied"])
fan, occ = w["fan-status"], w["occupied"]
flags = pd.DataFrame(
    {
        "equipment_id": w["equipment_id"],
        "run_unocc": (fan == 1) & (occ == 0),  # boolean masks
        "off_occ": (fan == 0) & (occ == 1),
        "occ": occ,
    }
)
g = flags.groupby("equipment_id")
unocc_pd = pd.DataFrame(
    {
        "run_unocc_h": g["run_unocc"].sum() * 5 / 60,
        "off_occ_h": g["off_occ"].sum() * 5 / 60,
        "pct_occupied": 100.0 * g["occ"].mean(),
    }
).reset_index()
lab.assert_parity(unocc_pd, unocc_sql, label="unoccupied run hours")
print(unocc_sql.round(1).to_string(index=False))
r1 = unocc_sql.set_index("equipment_id").round(1)
assert r1.loc["AHU_1"].tolist() == [58.6, 2.2, 51.3]
assert r1.loc["AHU_2", "off_occ_h"] == 30.8
```

(`CROSS JOIN` here only pins the join order, as on Day 8.) AHU_1 ran about 59 hours while the schedule said unoccupied. AHU_2 has the opposite pattern: 30.8 hours **off** while occupied, which is a comfort problem rather than an energy one. Note that `avg(o.value)` is "fraction of readings where occupied = 1", the same trick as `occ.mean()`.

Bucket outside air into bands and count. A multi-branch `CASE` is `np.select` (or `pd.cut` for numeric bins). The `WHEN`s are tried in order, so `value < 60` only sees values that already failed `value < 40`.

```
BANDS_SQL = """
SELECT CASE
         WHEN value < 40 THEN '1: below 40 F'
         WHEN value < 60 THEN '2: 40-60 F'
         WHEN value < 80 THEN '3: 60-80 F'
         ELSE '4: 80 F and up'
       END AS band,
       count(*) AS readings
FROM samples
WHERE timeseries_id = (SELECT timeseries_id FROM lab_point
                       WHERE equipment_id = 'AHU_1' AND openfdd_role =
         'outside-air-temp')
GROUP BY band
ORDER BY band
"""
bands_sql = lab.query(conn, BANDS_SQL)

oat = role_samples(["outside-air-temp"]).query("equipment_id ==
         'AHU_1'")["value"]
assert oat.notna().all()  # a NULL would fall into ELSE in SQL, and
         into `default` in np.select
labels = ["1: below 40 F", "2: 40-60 F", "3: 60-80 F"]
band = pd.Series(
    np.select([oat < 40, oat < 60, oat < 80], labels, default="4: 80 F
         and up"), name="band"
)
bands_pd =
         band.value_counts().rename("readings").rename_axis("band").reset_index(
lab.assert_parity(bands_pd, bands_sql, label="outside air bands")
print(bands_sql.to_string(index=False))
assert len(bands_sql) == 4

# the numeric-bin shortcut gives the same counts
cut = pd.cut(oat, [-np.inf, 40, 60, 80, np.inf], right=False,
         labels=labels + ["4: 80 F and up"])
assert cut.value_counts().sort_index().tolist() ==
         bands_sql["readings"].tolist()
```

Cost of the mask vs the Python `if`. The same “running while unoccupied” flag, three ways, on the 142,144 fan and occupancy rows:

```
def mask_way():
    return ((fan == 1) & (occ == 0)).sum()


def where_way():
    return np.where((fan == 1) & (occ == 0), 1, 0).sum()


def apply_way():
    return w.apply(lambda r: 1 if r["fan-status"] == 1 and
         r["occupied"] == 0 else 0, axis=1).sum()


t_mask, t_where, t_apply = (lab.best_time(f) for f in (mask_way,
         where_way, apply_way))
print(
    f"mask {t_mask * 1e3:7.2f} ms | np.where {t_where * 1e3:7.2f} ms |
         row-by-row apply {t_apply * 1e3:8.1f} ms"
)
assert mask_way() == where_way() == apply_way() ==
         round(unocc_pd.run_unocc_h.sum() * 60 / 5)
assert t_apply > 10 * t_mask
```

All three are O(n), but the Python-level loop pays interpreter overhead per row: 100× or more. Vectorized masks run in compiled code over the whole column. SQL's `CASE` runs in compiled code too, inside the scan.

Try it

Solve the week 1 challenge, now with `lab_point` and `CASE`: for each month, AHU_1's average outside air temperature, fan run hours, and readings with duct static above 1.5.

Answer

```
MONTHLY_SQL = """
SELECT strftime('%Y-%m', s.ts, 'unixepoch') AS month,
       avg(CASE WHEN p.openfdd_role = 'outside-air-temp' THEN s.value
         END)             AS avg_oat,
       sum(CASE WHEN p.openfdd_role = 'fan-status' THEN s.value ELSE 0
         END) * 5 / 60.0 AS run_h,
       sum(CASE WHEN p.openfdd_role = 'duct-static-pressure' AND
         s.value > 1.5 THEN 1 ELSE 0 END) AS high_static
FROM samples s
JOIN lab_point p ON p.timeseries_id = s.timeseries_id
WHERE p.equipment_id = 'AHU_1'
  AND p.openfdd_role IN ('outside-air-temp', 'fan-status', 'duct-
         static-pressure')
GROUP BY month
ORDER BY month
"""
monthly_sql = lab.query(conn, MONTHLY_SQL)

m = wide_roles(["outside-air-temp", "fan-status", "duct-static-
         pressure"])
m = m[m.equipment_id == "AHU_1"].assign(
    month=lambda d: pd.to_datetime(d["ts"], unit="s").dt.strftime("%Y-
         %m")
)
m["high"] = m["duct-static-pressure"] > 1.5
gm = m.groupby("month")
monthly_pd = pd.DataFrame(
    {
        "avg_oat": gm["outside-air-temp"].mean(),
        "run_h": gm["fan-status"].sum() * 5 / 60,
        "high_static": gm["high"].sum(),
    }
).reset_index()
lab.assert_parity(monthly_pd, monthly_sql, label="AHU_1 monthly
         summary")
print(monthly_sql.round(1).to_string(index=False))
assert round(monthly_sql.set_index("month").avg_oat["2026-03"], 1) ==
         41.5
```

One scan, three answers. The CASE picks which rows feed each column. In week 1 this took three queries, or a self-join. SQL keeps the data long (one row per reading, every role mixed together) and uses CASE to pick. pandas pivots to wide first, so each role is its own column. Same result, opposite shape. Day 11 stores the wide shape.

Why it matters

A fault rule is a CASE: *when* the fan runs at full speed *and* static is below setpoint *then* 1 *else* 0. In pandas it's a mask. Sum it and you have fault hours. Day 14 builds exactly that, both ways, with parity.

Day 10 — Time buckets and views: hourly averages (what Timescale's `time_bucket` does)

Goal: downsample 5-minute data to hourly in SQL and pandas, and save queries as views and summary tables.

Read

Time bucketing. Integer division rounds a timestamp down to a bucket:

```
ts / 3600 * 3600     -- start of the hour (3600 s)
ts / 900  * 900      -- start of the 15-minute bucket
ts / 86400 * 86400   -- start of the UTC day
```

`ts` is an integer, so `/` drops the remainder. Group by the bucket and aggregate, and you have hourly data. TimescaleDB, the Postgres extension many historians use, calls this `time_bucket('1 hour', time)`. SQLite does it with arithmetic.

pandas has two spellings of the same idea:

Idea	SQL	pandas
bucket start, by arithmetic	`ts / 3600 * 3600`	`ts // 3600 * 3600`
bucket start, on datetimes	`(datetime(ts / 3600 * 3600, 'unixepoch'))`	`dt.floor("h")`
bucket and aggregate	`GROUP BY ts / 3600`	`groupby(bucket)`, or `resample("1h", on="time")`
several aggregates	`avg(v), min(v), max(v), count(*)`	`.agg(mean="mean", lo="min", hi="max", n="size")`

Beware the integer division. In SQL, `/` on two integers truncates toward zero, which equals "round down" only for non-negative timestamps (every timestamp here). Python's `//` always floors, so the two would differ for dates before 1970. Neither bothers you today.

Each of these is a **hash aggregation**: one pass over the n rows, O(n), with a hash table holding one entry per bucket. SQL's `GROUP BY` on an expression can't use an index (the bucket is computed, not stored), so SQLite collects the groups in a temporary B-tree: O(n log b) for b buckets. `resample` instead needs sorted time, which a DataFrame sorted by `ts` already has.

Always keep a `count(*)` per bucket. A bucket with 4 readings instead of 12 is a partial hour, and its average is less trustworthy.

Views. `CREATE VIEW name AS SELECT ...` saves a query under a name. It stores no data; it reruns every time you use it. Good for not repeating yourself. The pandas analogue is a **function**: `reading(roles)` that loads and merges whenever you call it.

Summary tables. `CREATE TABLE name AS SELECT ...` runs the query once and stores the result. It's fast to read, but stale when new data arrives. In pandas it's a DataFrame you keep in a variable (or write to a file). Timescale's **continuous aggregates** are this idea maintained for you: a summary table it refreshes as new data lands. In SQLite you rebuild it yourself. The trade is **compute once, read many** (table) against **always fresh, pay every time** (view). We measure it below.

Units: raw (°F, inH2O, percent 0–100).

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()
lab.ensure_points(conn)
pts = lab.query(conn, "SELECT * FROM lab_point")
ROLE_COLS = pts[["timeseries_id", "equipment_id", "openfdd_role"]]
KEYS = ["equipment_id", "ts"]
ALL_ROLES = sorted(pts["openfdd_role"].dropna().unique())  # the 13
         roles: 26 points, 924k readings


def role_samples(roles):
    """Readings of just the points that carry one of these roles,
         labelled with equipment and role."""
    ids = pts.loc[pts.openfdd_role.isin(roles),
         "timeseries_id"].tolist()
    marks = ",".join("?" * len(ids))
    raw = lab.query(
        conn, f"SELECT timeseries_id, ts, value FROM samples WHERE
         timeseries_id IN ({marks})", ids
    )
    return raw.merge(ROLE_COLS, on="timeseries_id",
         validate="m:1").drop(columns="timeseries_id")
```

Hourly outside air temperature, AHU_1. First in SQL (the lookup in the `WHERE` finds the point by role, so no UUID is typed), then with arithmetic bucketing in pandas, then with `resample`.

```
HOURLY_SQL = """
SELECT ts / 3600 * 3600 AS hour_ts,
       avg(value) AS avg_f,
       count(*) AS n
FROM samples
WHERE timeseries_id = (SELECT timeseries_id FROM lab_point
                       WHERE equipment_id = 'AHU_1' AND openfdd_role =
         'outside-air-temp')
GROUP BY ts / 3600
ORDER BY ts / 3600
"""
hourly_sql = lab.query(conn, HOURLY_SQL)

oat = role_samples(["outside-air-temp"]).query("equipment_id ==
         'AHU_1'").sort_values("ts")
oat = oat.assign(hour_ts=oat["ts"] // 3600 * 3600)  # bucket start, by
         arithmetic
hourly_pd = oat.groupby("hour_ts")["value"].agg(avg_f="mean",
         n="size").reset_index()
lab.assert_parity(hourly_pd, hourly_sql, label="hourly OAT (arithmetic
         bucket)")

# resample works on a datetime index; convert back to epoch seconds to
         compare
by_time = oat.assign(time=pd.to_datetime(oat["ts"],
         unit="s")).set_index("time")["value"]
res = by_time.resample("1h").agg(["mean", "size"])
res_pd = pd.DataFrame(
    {
        "hour_ts": (res.index - pd.Timestamp("1970-01-01")) //
         pd.Timedelta("1s"),
        "avg_f": res["mean"].to_numpy(),
        "n": res["size"].to_numpy(),
    }
)
assert (res_pd.n > 0).all()  # resample would also emit empty hours;
         this historian has no gaps
lab.assert_parity(res_pd, hourly_sql, label="hourly OAT (resample)")

first = hourly_sql.iloc[0]
print(pd.to_datetime(first.hour_ts, unit="s"), round(first.avg_f, 2),
         int(first.n))
assert (pd.to_datetime(first.hour_ts, unit="s").isoformat(),
         round(first.avg_f, 2), first.n) == (
    "2026-03-16T00:00:00",
    69.03,
    4,
)
```

The first hour has only 4 readings (00:40–00:55), which is why `n` is kept. (`resample` fills in missing hours with `NaN` and a count of 0; SQL's `GROUP BY` only makes groups for hours that have data. Both agree here because the historian has no gaps.)

A view, and its pandas twin, a function. Every reading with its point attributes.

```
conn.execute("DROP VIEW IF EXISTS lab_reading")
conn.execute(
    """CREATE VIEW lab_reading AS
       SELECT p.equipment_id, p.openfdd_role, p.point_id, p.unit, s.ts,
         s.value
       FROM samples s
       JOIN lab_point p ON p.timeseries_id = s.timeseries_id"""
)

fan_sql = lab.query(
    conn,
    "SELECT equipment_id, count(*) AS n FROM lab_reading WHERE
         openfdd_role = 'fan-status' GROUP BY 1",
)
n_view = conn.execute(
    "SELECT count(*) FROM lab_reading WHERE openfdd_role = 'fan-
         status'"
).fetchone()[0]
fan_pd = role_samples(["fan-
         status"]).groupby("equipment_id").size().rename("n").reset_index()
lab.assert_parity(fan_pd, fan_sql, label="view vs function")
print("fan-status readings:", n_view)
assert n_view == 71072
```

`WHERE openfdd_role = 'fan-status'` on the view is pushed down into the view's own join, so SQLite still only touches the fan readings. The view stores nothing, and the next query joins again.

A summary table, and its pandas twin, a stored DataFrame. Hourly mean, min and max for every role point.

```
HOURLY_ALL_SQL = """
SELECT equipment_id, openfdd_role,
       ts / 3600 * 3600 AS hour_ts,
       avg(value) AS mean, min(value) AS lo, max(value) AS hi, count(*)
         AS n
FROM lab_reading
WHERE openfdd_role IS NOT NULL
GROUP BY equipment_id, openfdd_role, ts / 3600
"""
conn.execute("DROP TABLE IF EXISTS lab_hourly")
conn.execute("CREATE TABLE lab_hourly AS " + HOURLY_ALL_SQL)
conn.execute("CREATE INDEX lab_hourly_idx ON lab_hourly (equipment_id,
         openfdd_role, hour_ts)")
conn.commit()

allr = role_samples(ALL_ROLES)
allr["hour_ts"] = allr["ts"] // 3600 * 3600
hourly_all_pd = (
    allr.groupby(["equipment_id", "openfdd_role", "hour_ts"])["value"]
    .agg(mean="mean", lo="min", hi="max", n="size")
    .reset_index()
)
lab_hourly = lab.query(conn, "SELECT * FROM lab_hourly")
lab.assert_parity(
    hourly_all_pd, lab_hourly, keys=["equipment_id", "openfdd_role",
         "hour_ts"], label="lab_hourly"
)
print(len(lab_hourly), "rows in lab_hourly,", len(allr), "readings in
         the DataFrame")
assert len(lab_hourly) == 77012
```

26 role points × 2,962 hours = 77,012 rows, instead of 924k readings. Charts and dashboards read tables like this.

View vs table, measured. Ask the same question both ways: AHU_2's hourly duct static, as a daily maximum of the hourly mean. The view version recomputes the join and the bucketing every time. The table version reads rows that are already bucketed (and finds them through the index we just made).

```
ASK_VIEW = """
SELECT date(hour_ts, 'unixepoch') AS day, max(mean) AS peak
FROM (SELECT ts / 3600 * 3600 AS hour_ts, avg(value) AS mean
      FROM lab_reading
      WHERE equipment_id = 'AHU_2' AND openfdd_role = 'duct-static-
         pressure'
      GROUP BY ts / 3600)
GROUP BY day ORDER BY day
"""
ASK_TABLE = """
SELECT date(hour_ts, 'unixepoch') AS day, max(mean) AS peak
FROM lab_hourly
WHERE equipment_id = 'AHU_2' AND openfdd_role = 'duct-static-pressure'
GROUP BY day ORDER BY day
"""
via_view, via_table = lab.query(conn, ASK_VIEW), lab.query(conn,
         ASK_TABLE)
lab.assert_parity(via_view, via_table, label="same answer from the view
         and the table")

t_view = lab.best_time(lambda: conn.execute(ASK_VIEW).fetchall())
t_table = lab.best_time(lambda: conn.execute(ASK_TABLE).fetchall())
print(
    f"view {t_view * 1000:7.1f} ms | table {t_table * 1000:6.1f} ms |
         table is {t_view / t_table:.0f}x faster"
)
assert t_table < t_view
```

The view must scan all 35,536 AHU_2 static readings each time (O(n)). The table scans about 2,962 hourly rows via the index, about n/12 rows with no join. The cost the table hides is staleness:

```
conn.execute("CREATE TEMP TABLE t (x INTEGER)")
conn.execute("INSERT INTO t VALUES (1), (2)")
conn.execute("CREATE TEMP VIEW t_view AS SELECT sum(x) AS total FROM
         t")
conn.execute("CREATE TEMP TABLE t_table AS SELECT sum(x) AS total FROM
         t")
conn.execute("INSERT INTO t VALUES (3)")  # new data arrives
print(
    "view:",
    conn.execute("SELECT total FROM t_view").fetchone()[0],
    "| table:",
    conn.execute("SELECT total FROM t_table").fetchone()[0],
)
assert conn.execute("SELECT total FROM t_view").fetchone()[0] == 6
assert conn.execute("SELECT total FROM t_table").fetchone()[0] == 3  #
         stale until rebuilt
```

The view sees the new row at once; the table still says 3 until you rebuild it. Dashboards often accept that: rebuild hourly, read instantly. Timescale's continuous aggregates automate the rebuild.

Try it

1. From `lab_hourly`, the hour with AHU_2's highest average duct static pressure.
2. Write a 15-minute bucket version of the first query. How many readings does a full 15-minute bucket have?

Answers

```
TOP_SQL = """
SELECT hour_ts, mean
FROM lab_hourly
WHERE equipment_id = 'AHU_2' AND openfdd_role = 'duct-static-pressure'
ORDER BY mean DESC
LIMIT 1
"""
top_sql = lab.query(conn, TOP_SQL)
ahu2 = hourly_all_pd[
    (hourly_all_pd.equipment_id == "AHU_2") &
        (hourly_all_pd.openfdd_role == "duct-static-pressure")
]
top_pd = ahu2.nlargest(1, "mean")[["hour_ts", "mean"]]
lab.assert_parity(top_pd, top_sql, label="AHU_2 highest hourly static")
print(pd.to_datetime(top_sql.hour_ts.iloc[0], unit="s"),
        round(top_sql["mean"].iloc[0], 3))
assert len(top_sql) == 1


QUARTER_SQL = """
SELECT ts / 900 * 900 AS bucket, count(*) AS n
FROM samples
WHERE timeseries_id = (SELECT timeseries_id FROM lab_point
                       WHERE equipment_id = 'AHU_1' AND openfdd_role =
        'outside-air-temp')
GROUP BY ts / 900 ORDER BY ts / 900
"""
quarter_sql = lab.query(conn, QUARTER_SQL)
quarter_pd = (
    oat.assign(bucket=oat["ts"] // 900 *
        900).groupby("bucket").size().rename("n").reset_index()
)
lab.assert_parity(quarter_pd, quarter_sql, label="15-minute buckets")
print(
    quarter_sql.head(3)
    .assign(bucket=lambda d: pd.to_datetime(d.bucket, unit="s"))
    .to_string(index=False)
)
assert quarter_sql.n.iloc[:3].tolist() == [1, 3, 3]
```

A full 15-minute bucket has 3 readings. The first has only one, at 00:40 (the 00:30 bucket).

Week 2 challenge

Build `lab_daily`: one row per AHU per day, with fan run hours, hours run while unoccupied, average outside air temperature, and average static pressure *while running*. Then find the five days with the most unoccupied runtime on AHU_1, and check the day of week (`strftime('%w', ...)`): weekends?

Answer

SQL first. It pivots the roles into columns with the conditional aggregation of Day 9, then buckets by day. pandas does the same with a `pivot` and a `groupby` on the day.

```
DAILY_SQL = """
WITH wide AS (
  SELECT p.equipment_id, s.ts,
         max(CASE WHEN p.openfdd_role = 'fan-status' THEN s.value END)
         AS fan,
         max(CASE WHEN p.openfdd_role = 'occupied' THEN s.value END)
         AS occupied,
         max(CASE WHEN p.openfdd_role = 'outside-air-temp' THEN s.value
         END)     AS oat,
         max(CASE WHEN p.openfdd_role = 'duct-static-pressure' THEN
         s.value END) AS static
  FROM samples s
  JOIN lab_point p ON p.timeseries_id = s.timeseries_id
  WHERE p.openfdd_role IN ('fan-status', 'occupied', 'outside-air-
         temp', 'duct-static-pressure')
  GROUP BY p.equipment_id, s.ts
)
SELECT equipment_id,
       date(ts, 'unixepoch')
         AS day,
       sum(fan) * 5 / 60.0
         AS run_h,
       sum(CASE WHEN fan = 1 AND occupied = 0 THEN 1 ELSE 0 END) * 5 /
         60.0    AS run_unocc_h,
       avg(oat)
         AS avg_oat,
       avg(CASE WHEN fan = 1 THEN static END)
         AS avg_static_running
FROM wide
GROUP BY equipment_id, day
"""
conn.execute("DROP TABLE IF EXISTS lab_daily")
conn.execute("CREATE TABLE lab_daily AS " + DAILY_SQL)
conn.commit()
daily_sql = lab.query(conn, "SELECT * FROM lab_daily")

d = role_samples(["fan-status", "occupied", "outside-air-temp", "duct-
         static-pressure"])
d = d.pivot(index=KEYS, columns="openfdd_role",
         values="value").reset_index()
d["day"] = pd.to_datetime(d["ts"], unit="s").dt.strftime("%Y-%m-%d")
d["run_unocc"] = (d["fan-status"] == 1) & (d["occupied"] == 0)
d["static_running"] = d["duct-static-pressure"].where(
    d["fan-status"] == 1
)  # NaN when off, skipped by mean
gd = d.groupby(["equipment_id", "day"])
daily_pd = pd.DataFrame(
    {
        "run_h": gd["fan-status"].sum() * 5 / 60,
        "run_unocc_h": gd["run_unocc"].sum() * 5 / 60,
        "avg_oat": gd["outside-air-temp"].mean(),
        "avg_static_running": gd["static_running"].mean(),
    }
).reset_index()
lab.assert_parity(daily_pd, daily_sql, keys=["equipment_id", "day"],
         label="lab_daily")
```

```
print(len(daily_sql), "rows:",
          daily_sql.groupby("equipment_id").size().to_dict())
assert len(daily_sql) == 2 * 124  # 124 days (Mar 16 to Jul 17) for
          each AHU
```

Now the five days with the most unoccupied runtime on AHU_1, and their weekday. SQLite's `%w` counts Sunday as 0; pandas' `dayofweek` counts Monday as 0, so convert before comparing.

```
TOP5_SQL = """
SELECT day, run_unocc_h, strftime('%w', day) AS dow
FROM lab_daily
WHERE equipment_id = 'AHU_1'
ORDER BY run_unocc_h DESC, day
LIMIT 5
"""
top5_sql = lab.query(conn, TOP5_SQL)

a1 = (
    daily_pd[daily_pd.equipment_id == "AHU_1"]
    .sort_values(["run_unocc_h", "day"], ascending=[False, True])
    .head(5)
)
sunday_first = (pd.to_datetime(a1["day"]).dt.dayofweek + 1) % 7
top5_pd = a1.assign(dow=sunday_first.astype(str))[["day",
          "run_unocc_h", "dow"]]
lab.assert_parity(top5_pd, top5_sql, keys=["day"], label="top 5
          unoccupied days")

NAMES = {"0": "Sun", "1": "Mon", "2": "Tue", "3": "Wed", "4": "Thu",
          "5": "Fri", "6": "Sat"}
print(top5_sql.assign(weekday=top5_sql.dow.map(NAMES)).to_string(index=False))
```

Not weekends. The heaviest days are Wednesday 2026-05-20 (10.1 h running while unoccupied), Tuesday 2026-04-14 and Monday 2026-05-18, with only one Sunday (2026-05-17) in the top five. The extra unoccupied runtime sits on weekdays: more likely a schedule that starts the fan early or stops it late than a weekend left running. (The `ORDER BY run_unocc_h DESC, day` tie-break matters: 2026-05-17 and 2026-06-29 both have 9.17 h.) That's a lead for a day-by-day look at the occupancy command and fan status.

Reading — Week 2

The lessons first, then these for exact behavior.

- SQLite joins (SELECT, the FROM clause)
- SQLite CASE expressions
- SQLite CREATE VIEW
- DB Browser: importing CSV
- TimescaleDB time_bucket